Cache Invalidation

1. What is Cache Invalidation?

Cache invalidation is the process of removing, updating, or replacing old cached data when the original data changes.

Goal:

```
None
Prevent stale data

Serve fresh responses

Maintain consistency
```

If cache is not invalidated, users may see outdated data.

2. Simple Example

Database value:

```
None
Product Price = ₹999
```

Cached value:

```
None
Product Price = ₹999
```

Later database updated:

```
None
Product Price = ₹799
```

If cache not invalidated:

```
None
Users still see ₹999
```

Wrong result.

3. Why It Is Hard

Caching improves speed, but creates duplicate copies of data.

Now data exists in:

```
None
Database

Redis

CDN

Browser Cache

Application Memory
```

All copies must stay reasonably fresh.

That makes invalidation difficult.

4. Common Cache Invalidation Methods

A. TTL (Time To Live)

Auto-expire data after time.

```
None
Key expires in 5 minutes
```

After expiry, next request reloads fresh data.

Best for:

```
None
Product pages

Public APIs

Frequently changing data
```

B. Delete on Write

When DB updates, remove cache key.

```
None
UPDATE users SET name='Sam'

DELETE cache:user:44
```

Next read fetches fresh value.

Most common strategy.

C. Update Cache on Write

When DB updates, immediately update cache too.

```
None
DB updated

Cache updated same time
```

Used in write-through caching.

D. Versioned Keys

Instead of deleting old cache, create new version.

```
None
profile:v1

profile:v2
```

Old versions naturally expire.

Useful for static assets.

5. Cache Busting (For Files)

Cache busting = cache invalidation specifically for files like:

```
None
CSS

JavaScript

Images

Fonts
```

Example:

Old file:

```
None
app.css
```

New deployment:

```
None
app.v2.css

or

app.css?hash=abc123
```

Browser treats as new file and downloads fresh copy.

6. Example Website Deployment

Without cache busting:

```
None
User still loads old JS file

Broken UI may happen
```

With cache busting:

```
None
main.7fa12.js

main.9bd77.js
```

Fresh version loaded instantly.

7. Push vs Pull Invalidation

Push

System actively removes cache entries.

```
None
DB update triggers Redis delete
```

Pull

Cache expires naturally via TTL.

```
None
Wait until timeout
```

8. Multi-Layer Cache Invalidation

Need to clear multiple layers:

```
Browser

CDN

API Gateway

Redis

Local Memory Cache
```

Example:

Product image changed → clear CDN + browser + backend cache.

9. Common Problems

A. Stale Reads

```
User sees old profile info
```

B. Race Conditions

Two updates happen quickly.

```
Old value overwrites new cache
```

C. Thundering Herd

Many users request same key after expiry.

```
None
All hit DB simultaneously
```

10. Solutions

Use:

```
None
Distributed locks

Request coalescing

Jittered TTL

Background refresh

Version numbers
```

11. Real-World Examples

E-Commerce

```
Price changed → invalidate product cache

Stock changed → invalidate inventory cache
```

Social Media

```
User changed profile pic → invalidate profile cache
```

News Site

```
Breaking article edited → purge CDN cache
```

12. Best HLD Strategy

For most systems:

```
Read-heavy data → TTL + delete-on-write

Critical data → immediate invalidate

Static files → versioned cache busting
```

13. Interview Talking Points

If asked hardest part of caching:

```
There are only two hard things:

Cache invalidation

Naming things

Off-by-one errors
```

(Well-known engineering joke)

14. Advantages of Proper Invalidation

```
None
```

```
Fresh data

Fast responses

Consistent UX

Lower DB load
```

15. Disadvantages / Tradeoffs

```
None
```

```
Operational complexity

Extra code paths

Event ordering issues

More infrastructure logic
```

16. One-Line Summary

Cache invalidation is the process of removing or refreshing outdated cached data so users get fresh results while still benefiting from caching speed. Cache busting is the same concept applied to static files.